

RAPPORT DE PROJET — MODULE BIG DATA

Système de Recommandation en Temps Réel Un Pipeline Big Data Inspiré du Modèle Lambda

Apache Kafka · Apache Spark · Filtrage Collaboratif ALS · FastAPI

RÉALISÉ PAR

El Gorrin Mohamed

Ingénieur ML — Modèle ALS et Décomposition des Biais

Kchibal Ismail

Ingénieur Data — Kafka, Streaming Spark

Uthman Junaid

Full-Stack — Dashboard, API et Orchestration

ENCADRÉ PAR

Pr. Yassin EL YUSUFI

Table des matières

1	Problématique	2
1.1	Caractéristiques du jeu de données	2
1.2	Défis identifiés	2
2	Architecture	2
3	Flux d'exécution étape par étape	3
3.1	Étape 1 — Ingestion des données (<i>Kchibal Ismail</i>)	3
3.2	Étape 2 — Entraînement batch (<i>El Gorrin Mohamed</i>)	4
3.3	Étape 3 — Inférence en streaming (<i>Kchibal Ismail</i>)	6
3.4	Étape 4 — Stockage (PostgreSQL)	7
3.5	Étape 5 — Orchestration, API et tableau de bord (<i>Uthman Junaid</i>)	7
4	Lecture du tableau de bord	9
4.1	Fiche utilisateur	9
4.2	Cartes de recommandation	9
4.3	Liste des utilisateurs indexés	10
5	Résultats	10
5.1	Performance du modèle	10
5.2	Échelle et métriques système	11
6	Déploiement	11
6.1	Prérequis	11
6.2	Démarrage en une commande	11
7	Limites	11
8	Perspectives d'amélioration	12
9	Contributions	12
	Conclusion	13

1 Problématique

Ce projet construit un système de recommandation de produits en temps réel à partir du jeu de données Amazon Fine Food Reviews — une collection publique de 568 454 avis rédigés entre 1999 et 2012.

1.1 Caractéristiques du jeu de données

Attribut	Valeur
Nombre d'avis	568 454
Utilisateurs uniques	256 059
Produits uniques	74 258
Note moyenne globale	4,37
Note médiane	5,0
Taux de sparsité	99,9971%
Période couverte	1999 – 2012

TABLE 1. Résumé du jeu de données Amazon Fine Food Reviews.

1.2 Défis identifiés

Biais positif extrême. 63,9% de l'ensemble des notes sont des 5 étoiles. Une baseline naïve qui prédit la moyenne globale pour chaque paire obtient un RMSE $\approx 1,55$. L'ALS standard entraîné sur les notes brutes hérite de ce biais : les facteurs latents consacrent leur capacité à modéliser la position sur l'échelle de notation plutôt que les préférences réelles.

Sparsité. 99,9971% des paires utilisateur-produit n'ont aucune interaction. 68,5% des utilisateurs n'ont rédigé qu'un seul avis. Le filtrage collaboratif se dégrade dans ce régime.

Étiquettes bruitées. Environ 15% des avis affichent un ratio d'utilité inférieur à 0,6 — la communauté les a jugés peu fiables.

Démarrage à froid. Les utilisateurs absents de l'ensemble d'entraînement ne peuvent pas recevoir de recommandation personnalisée d'un modèle de factorisation matricielle. Une stratégie de repli est nécessaire.

Contrainte temps réel. Le système doit s'entraîner hors ligne tout en diffusant simultanément les événements en direct vers un tableau de bord de supervision, ce qui exige une architecture à deux couches.

2 Architecture

Le pipeline suit une découpe inspirée du modèle Lambda : une *couche batch* entraîne le modèle sur un instantané historique ; une *couche vitesse* applique le modèle figé aux nouveaux événements en temps réel.

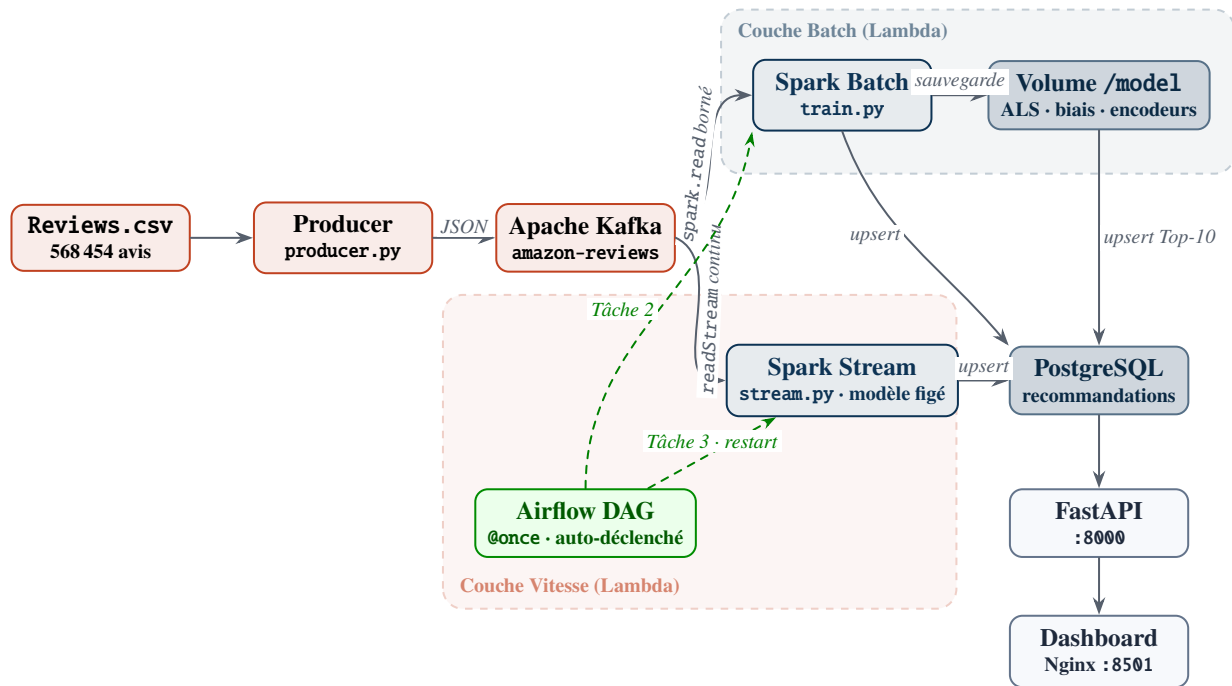


FIGURE 1. Architecture Lambda-inspirée : couche batch (entraînement ALS hors ligne sur instantané borné) et couche vitesse (inférence en streaming temps réel sur flux continu). Les deux couches partagent le volume /model et la base PostgreSQL, orchestrées par un unique DAG Airflow.

Les deux couches partagent un volume Docker situé à /model. Après le réentraînement, Airflow exécute `docker restart spark-stream`, ce qui provoque le rechargement automatique du nouveau modèle depuis le disque par le service de streaming.

3 Flux d'exécution étape par étape

Cette section suit l'ordre d'exécution du pipeline, des données brutes aux recommandations en direct.

3.1 Étape 1 — Ingestion des données (Kchibal Ismail)

3.1.1 Rôle de Kafka

Apache Kafka est un courtier de messages distribué. Il joue le rôle de file d'attente persistante : les producteurs y écrivent des messages, les consommateurs les lisent. Kafka conserve chaque message pendant une durée configurable, ce qui permet au job d'entraînement batch et au service d'inférence en streaming de lire les mêmes données de façon indépendante, à leur propre rythme, sans coordination entre eux.

3.1.2 Infrastructure mise en place

Kchibal Ismail a configuré le cluster Kafka via Docker Compose :

- **ZooKeeper** : service de coordination requis par Kafka pour l'enregistrement des brokers et l'élection du leader.
- **Broker Kafka** : service de stockage et de routage des messages, accessible sur le port 29092 (interne) et 9092 (externe).
- **kafka-init** : conteneur à usage unique qui crée le topic `amazon-reviews` avec 1 partition et un facteur de réplication de 1, puis se termine.

Des vérifications de santé (*health checks*) sont définies sur chaque service afin que les conteneurs dépendants (Airflow, Spark, API) ne démarrent qu’une fois leurs dépendances prêtes.

3.1.3 Le producteur (`producer.py`)

Le producteur est un script Python qui lit `Reviews.csv` ligne par ligne et publie chaque avis sous forme de message JSON dans Kafka. Il s’exécute en tant que **Tâche 1 d’Airflow** — et non comme un service Docker autonome. Cette décision de conception est délibérée : le producteur s’exécutant de façon synchrone dans Airflow, la Tâche 2 (entraînement Spark) ne peut démarrer qu’après que tous les messages sont dans Kafka. Il n’y a ainsi aucun risque de lancer l’entraînement sur un jeu de données incomplet.

Chaque message Kafka contient :

```

1 {
2   "UserId":           "A3SGXH7AUHU8GW",
3   "ProductId":        "B001E4KFG0",
4   "Score":            4.0,
5   "Time":             1346976000,
6   "HelpfulnessNumerator": 1,
7   "HelpfulnessDenominator": 1,
8   "Summary":          "Good Quality Dog Food"
9 }
```

L’exécution complète envoie 568 454 messages à pleine vitesse (`DELAY_MS=0`) et se termine en environ 30 minutes.

3.2 Étape 2 — Entraînement batch (*El Gorrim Mohamed*)

Une fois l’ingestion terminée, Airflow déclenche la Tâche 2 : `spark-submit train.py`. C’est le cœur du pipeline ML.

3.2.1 Lecture depuis Kafka (instantané borné)

`train.py` utilise `spark.read.format("kafka")` avec `startingOffsets="earliest"` et `endingOffsets="latest"`. Il s’agit d’une lecture *bornée* — Spark lit tous les messages présents au moment du démarrage du job, puis s’arrête. Il n’attend pas de nouveaux messages. Le résultat est un `DataFrame` statique de l’ensemble des 568 454 avis.

3.2.2 Nettoyage et filtrage

Les données brutes sont nettoyées en cinq étapes :

1. **Suppression des valeurs nulles** : élimination des lignes où `UserId`, `ProductId` ou `Score` est absent.
2. **Filtre d’intervalle** : conservation des notes comprises dans `[1, 5]`.
3. **Filtre d’utilité** : conservation des avis dont le dénominateur d’utilité est 0 (aucun vote, donc aucun signal de mauvaise qualité) OU dont `HelpfulnessNumerator/HelpfulnessDenominator` $\geq 0,6$. Ce filtre supprime environ 15% des avis jugés peu fiables par la communauté.
4. **Déduplication** : conservation d’une seule note par paire (`UserId`, `ProductId`). Si un utilisateur a évalué le même produit deux fois, seule la première entrée est conservée.
5. **Filtre d’activité** : suppression des utilisateurs ayant moins de 3 notes et des produits ayant moins de 3 notes, afin de garantir un signal suffisant pour le filtrage collaboratif.

Après filtrage, le jeu de données passe de 568 454 à **235 831** notes qualifiées, couvrant **40 456 utilisateurs** et **13 494 produits**.

3.2.3 Encodage des identifiants

L'ALS requiert des identifiants entiers pour les utilisateurs et les produits. Le `StringIndexer` de Spark MLlib associe chaque chaîne `UserId` unique à un entier séquentiel `userId`, et chaque `ProductId` à `productId`. Le pipeline d'encodeurs ajusté est sauvegardé dans `/model/encoders` pour être réutilisé lors de l'inférence en streaming.

3.2.4 Découpage train / validation / test — avant le calcul des biais

Le jeu de données filtré est découpé en 80% / 10% / 10% via `randomSplit([0.8, 0.1, 0.1], seed=42)`.

Ce découpage doit impérativement précéder tout calcul de biais. Calculer les biais sur l'ensemble complet (y compris les lignes de validation et de test) avant de diviser constitue une fuite de données : les statistiques de biais dérivées des lignes de test s'infiltreraient dans l'évaluation du modèle. Lors du développement, cette erreur a été commise et a produit un RMSE de validation anormalement bas de 0,24. Une fois l'ordre corrigé, des métriques honnêtes ont été obtenues.

3.2.5 Décomposition des biais — l'innovation clé

L'intuition centrale de ce projet est que les données de notation sont dominées par les biais d'échelle, et non par un signal de préférence réelle. Chaque note peut s'écrire :

$$r_{ui} = \mu + b_u + b_i + \varepsilon_{ui}$$

- μ — la note moyenne globale sur l'ensemble des avis d'entraînement (valeur trouvée : 4,37).
- b_u — le biais utilisateur : de combien l'utilisateur u note au-dessus ou en dessous de μ en moyenne. Un utilisateur habitué aux 5 étoiles a un b_u fortement positif; un utilisateur critique a un b_u négatif.
- b_i — le biais produit : de combien le produit i est noté au-dessus ou en dessous de μ en moyenne. Un produit constamment bien évalué a un b_i positif.
- ε_{ui} — le résidu : la part de la note qui ne peut s'expliquer ni par la tendance globale, ni par les habitudes de l'utilisateur, ni par la qualité du produit. C'est le véritable signal de préférence.

Les trois statistiques (μ , b_u , b_i) sont calculées *uniquement à partir du jeu d'entraînement*. L'ALS s'entraîne ensuite sur les résidus ε . À l'inférence :

$$\hat{r}_{ui} = \hat{\varepsilon}_{ui} + \mu + b_u + b_i \quad \text{borné à } [1,0, 5,0]$$

Les résidus étant centrés près de zéro et pouvant être négatifs, l'ALS est configuré avec `nonnegative=False` et `ratingCol="residual"`.

3.2.6 Recherche d'hyperparamètres

Le `TrainValidationSplit` de Spark MLlib effectue une recherche exhaustive sur 6 combinaisons avec 2 travailleurs parallèles :

Paramètre	Valeurs testées	Meilleure valeur
rank	{20, 50, 100}	100
regParam	{0.05, 0.1}	0.05
maxIter	15	15

TABLE 2. Grille d'hyperparamètres et meilleure configuration trouvée.

`TrainValidationSplit` utilise un découpage interne 80/20 de `train_df` pour sélectionner le meilleur modèle. Celui-ci est ensuite évalué *une seule fois* sur l'ensemble de test retenu, qui n'a jamais été vu pendant l'entraînement ni le réglage. Résultats :

- RMSE de validation : **0,6808**
- RMSE de test : **0,6764** (écart de 0,004 — aucun surapprentissage significatif)

3.2.7 Artefacts produits

Après l'entraînement, les artefacts suivants sont sauvegardés dans le volume partagé `/model` et dans PostgreSQL :

- `/model/als` — l'ALSModel entraîné
- `/model/encoders` — le pipeline `StringIndexer` ajusté
- `/model/user_biases` — Parquet : une ligne par utilisateur avec sa valeur b_u
- `/model/item_biases` — Parquet : une ligne par produit avec sa valeur b_i
- `/model/metrics.json` — RMSE, meilleurs paramètres, moyenne globale, nombre de lignes, horodatages
- `/model/popular_products.json` — Top-10 des produits les plus évalués (repli démarrage à froid)
- PostgreSQL recommendations — Top-10 prédictions pour les 40 456 utilisateurs connus
- PostgreSQL user_biases et item_biases — valeurs de biais pour les décompositions de scores dans le tableau de bord
- PostgreSQL products — noms d'affichage des 74 258 produits

Durée totale d'entraînement sur le jeu de données complet de 568 454 messages : **environ 20 minutes**.

3.3 Étape 3 — Inférence en streaming (*Kchibal Ismail*)

3.3.1 À quoi sert le service de streaming ?

L'étape d'entraînement batch génère des recommandations pour tous les utilisateurs présents dans l'ensemble d'entraînement au moment du job. Mais de nouveaux utilisateurs continuent d'arriver dans Kafka. Le service de streaming les prend en charge en temps réel, sans réentraîner le modèle.

`stream.py` est un service Docker toujours actif (`restart: unless-stopped`) qui charge le modèle ALS figé une seule fois au démarrage, puis l'applique en continu aux événements Kafka entrants.

3.3.2 Fonctionnement — micro-lots de 30 secondes

Le job de streaming utilise `spark.readStream.format("kafka")` — une lecture *non bornée* qui ne s'arrête jamais. Spark traite les nouveaux messages par lots toutes les 30 secondes via le patron `foreachBatch` :

1. Collecte de toutes les valeurs `UserId` uniques du lot Kafka des 30 dernières secondes.
2. Vérification des utilisateurs vus pendant l'entraînement (présents dans le vocabulaire de l'encodeur).
3. **Utilisateurs connus** : appel à `als_model.recommendForUserSubset(user_subset, TOP_N)`, application de la correction de biais ($\hat{r} = \hat{\epsilon} + \mu + b_u + b_i$) et mise à jour (*upsert*) dans PostgreSQL.
4. **Utilisateurs en démarrage à froid** (absents de l'entraînement) : ces utilisateurs ne sont *pas* écrits en base. L'API leur sert dynamiquement des replis sur produits populaires depuis `popular_products.json` au moment de la requête.

Un point de reprise (*checkpoint*) dans `/model/stream_checkpoint` garantit que, si le service redémarre, il reprend là où il s'était arrêté sans retraiter les anciens messages.

3.3.3 Pourquoi un modèle figé ?

L'ALS est un algorithme batch — il ne peut pas être mis à jour de façon incrémentale. Le service de streaming ne réentraîne pas ; il se contente d'*appliquer* les poids appris lors de l'entraînement batch. Lorsqu'un nouvel entraînement se termine (Tâche 3 Airflow : `docker restart spark-stream`), le service de streaming recharge le nouveau modèle depuis le disque. En attendant, il continue d'utiliser le modèle précédent.

3.4 Étape 4 — Stockage (PostgreSQL)

Toutes les données de recommandation sont stockées dans PostgreSQL. Le schéma est conçu pour une recherche rapide et des mises à jour sans conflit :

Table	Contenu
recommendations	(user_id, product_id, rank, predicted_rating) — UNIQUE sur (user_id, product_id)
user_biases	(user_id, user_bias) — une ligne par utilisateur entraîné
item_biases	(product_id, item_bias) — une ligne par produit entraîné
products	(product_id, display_name, review_count) — 74 258 produits
model_runs	(val_rmse, test_rmse, best_rank, train_rows, ...) — historique

TABLE 3. Schéma PostgreSQL.

Les mises à jour (*upserts*) évitent les violations de clé dupliquée grâce au patron de table de transit : les résultats sont écrits dans une table temporaire via JDBC, puis fusionnés dans la table principale avec `INSERT ... ON CONFLICT DO UPDATE`, avant que la table temporaire soit supprimée. Cela garantit l'atomicité des opérations.

Après un entraînement complet, la base contient **398 730 lignes de recommandations**, toutes issues de prédictions ALS réelles, sans bruit provenant du démarrage à froid.

3.5 Étape 5 — Orchestration, API et tableau de bord (Uthman Junaid)

3.5.1 Orchestration — DAG Airflow

Uthman Junaid a conçu et implémenté le DAG Airflow `recommendation_pipeline` qui enchaîne les quatre étapes :

Tâche	Type	Action
trigger_kafka_ingestion	BashOperator	Lance <code>producer.py</code> — 568 454 messages
spark_batch_training	BashOperator	<code>spark-submit train.py</code>
restart_streaming_service	BashOperator	<code>docker restart spark-stream</code>
print_model_metrics	PythonOperator	Lit <code>metrics.json</code> , journalise le RMSE

TABLE 4. Chaîne de tâches du DAG. Exécution séquentielle.

Le DAG est configuré avec `schedule='@once'`, `catchup=False` et `is_paused_upon_creation=False`. Il se déclenche automatiquement au démarrage de la stack — aucune intervention manuelle n'est nécessaire. Durée totale du pipeline : **33 minutes**.

3.5.2 Endpoints REST FastAPI

FastAPI expose les endpoints suivants, soutenus par un pool de connexions psycopg2 :

```
1 GET /recommendations/user/{user_id}?n=10
2   Retourne le Top-N avec predicted_rating, user_bias,
3   item_bias, als_residual pour chaque article.
4   Les utilisateurs inconnus reçoivent des replis sur
5   produits populaires (sans écriture en base).
6
7 GET /users?limit=100
8   Retourne un échantillon aléatoire d'utilisateurs
9   avec de vraies predictions ALS.
10  Les utilisateurs en démarrage à froid sont exclus.
11
12 GET /stats
13   Retourne : personalized_users, cold_start_users,
14             total_rows.
15
16 GET /metrics
17   Retourne : val_rmse, test_rmse, best_params,
18             global_mean, train_rows,
19             unique_users, unique_products.
20
21 GET /health/all
22   Retourne l'état des composants : postgres, kafka,
23   spark_training, spark_streaming.
24
25 GET /feed/latest?after=<seq>
26   Endpoint de scrutation : retourne au plus 10
27   événements Kafka récents avec un numéro de
28   séquence supérieur à <after>.
```

3.5.3 Flux en direct — architecture par scrutation

Le tableau de bord affiche un flux en direct des avis Kafka entrants. L'API maintient un tampon circulaire des 200 événements Kafka les plus récents (via un deque(maxlen=200) Python) alimenté par une tâche asyncio en arrière-plan. Chaque événement reçoit un numéro de séquence monotone.

Le tableau de bord interroge GET /feed/latest?after=<last_seq> toutes les 3 secondes. Ce choix de scrutation est délibérément préféré aux SSE (Server-Sent Events) : les SSE provoquaient des crashes mémoire dans le navigateur lors d'une ingestion Kafka rapide, car Chrome met en tampon l'intégralité du corps de la réponse HTTP en mémoire. La scrutation élimine ce problème : chaque réponse HTTP est petite, bornée et immédiatement libérée.

3.5.4 Tableau de bord

Le tableau de bord est une application monopage (SPA) servie par Nginx, qui fait office de proxy inverse entre /api/ et FastAPI. Il comprend cinq sections :

Section	Contenu
Flux en direct	Flux défilant d'événements Kafka ; résumé, ASIN, note
Recommandations	Top-N pour tout utilisateur avec décomposition complète du score
Analytique modèle	Historique RMSE, tableau des hyperparamètres, résumé d'entraînement
EDA du jeu de données	Distribution des notes, analyse de sparsité, motivation des biais
Santé du pipeline	État en direct des quatre composants du système

TABLE 5. Sections du tableau de bord.

4 Lecture du tableau de bord

Cette section décrit chaque élément de l'interface de recommandations afin de permettre une démonstration claire du système.

4.1 Fiche utilisateur

Lorsqu'un identifiant utilisateur est soumis, le tableau de bord affiche un bandeau de profil :

```
A3KX41MYHDSMI6  ✓ Utilisateur connu
10 Recommandations  4,09 Moy. prédite  -0,37 Biais utilisateur  4,37
Moyenne globale
```

- **Utilisateur connu** : cet utilisateur est apparu dans l'ensemble d'entraînement avec ≥ 3 avis. Le modèle ALS a appris des facteurs latents pour lui. Un « Utilisateur inconnu » recevrait des replis sur produits populaires.
- **10 Recommandations** : nombre de résultats Top-N affichés (paramétrable : Top 5 ou Top 10).
- **4,09 Moy. prédite** : moyenne des 10 notes prédites pour cet utilisateur.
- **-0,37 Biais utilisateur** (b_u) : cet utilisateur note les produits 0,37 étoile en dessous de la moyenne globale. C'est un évaluateur légèrement critique. Cette valeur réelle est stockée dans la table `user_biases`, calculée uniquement sur les données d'entraînement.
- **4,37 Moyenne globale** (μ) : la baseline — la note moyenne sur l'ensemble des 235 831 lignes d'entraînement.

4.2 Cartes de recommandation

Chaque carte de recommandation affiche :

```
[P4] #1 B00126GBP4
"bacon... Bacon... BACON!!!!!!"
***** 3,58 / 5,00
```

- **[P4]** : les deux derniers caractères de l'ASIN Amazon, utilisés comme initiales de la vignette. La couleur est déterministe — dérivée d'un hachage de l'ASIN — de sorte que le même produit affiche toujours la même couleur.
- **#1** : le rang de cette recommandation (1 = note prédite la plus élevée).
- **B00126GBP4** : l'Amazon Standard Identification Number (ASIN). C'est le véritable identifiant produit dans le jeu de données.

- **"bacon... Bacon... BACON!!!!!!"** : le résumé de l'évaluateur le plus utile pour ce produit, utilisé comme libellé d'affichage. Le jeu de données ne contient pas de noms de produits officiels — uniquement du texte rédigé par les évaluateurs. Il s'agit d'une limitation du jeu de données ; l'ASIN reste l'identifiant faisant autorité.
- **3,58 / 5,00** : la note prédite $\hat{r}_{ui} = \hat{\varepsilon}_{ui} + \mu + b_u + b_i$, bornée à $[1,0, 5,0]$. Les étoiles sont arrondies pour l'affichage.

Un clic sur « Décomposition du score » d'une carte révèle la décomposition exacte :

$$\hat{r}_{ui} = \underbrace{\mu}_{4,370} + \underbrace{b_u}_{-0,370} + \underbrace{b_i}_{\text{+biais produit}} + \underbrace{\hat{\varepsilon}_{ui}}_{\text{signal ALS}} = 3,58$$

Les quatre valeurs sont des nombres réels récupérés depuis PostgreSQL (tables `user_biases`, `item_biases`) et calculés par l'API — rien n'est approximé.

4.3 Liste des utilisateurs indexés

Le panneau « Utilisateurs indexés » charge un échantillon aléatoire d'identifiants utilisateurs via `GET /users?limit=100`. Ce sont des utilisateurs pour lesquels le modèle ALS a appris de véritables préférences — ils avaient au moins 3 notes dans l'ensemble d'entraînement. La liste change à chaque actualisation car la requête utilise `ORDER BY RANDOM()`. Les utilisateurs en démarrage à froid (moins de 3 avis) sont exclus de cette liste ; ils recevraient des replis sur produits populaires s'ils étaient interrogés.

Un clic sur un identifiant de cette liste lance une requête de recommandation pour cet utilisateur. Comme tous les utilisateurs listés sont connus du modèle, le résultat affichera toujours un profil complet avec le biais utilisateur, la note prédite moyenne et les scores ALS réels.

5 Résultats

5.1 Performance du modèle

Configuration	RMSE test	Remarques
Prédicteur moyenne globale	$\approx 1,55$	Prédit μ pour chaque paire
ALS brut (sans décomposition)	1,03 – 1,40	ALS standard, sans prétraitement
ALS + biais (instantané 200k)	0,4975	Exécution antérieure, données partielles
ALS + biais (568k complet)	0,6764	Exécution finale — 40 456 utilisateurs

TABLE 6. Comparaison des RMSE. Tous calculés sur l'ensemble retenu de 10%.

L'exécution finale (jeu de données complet) atteint un RMSE de validation de **0,6808** et un RMSE de test de **0,6764**, avec un écart de seulement 0,004 — confirmant l'absence de surapprentissage significatif.

Le RMSE plus élevé sur le jeu de données complet par rapport à l'instantané à 200k est attendu : le jeu complet inclut davantage d'utilisateurs peu actifs (ceux avec exactement 3 avis), qui sont plus difficiles à modéliser avec précision.

5.2 Échelle et métriques système

Métrique	Valeur
Messages envoyés dans Kafka	568 454
Lignes après tous les filtres	235 831
Lignes d'entraînement (80%)	188 773
Utilisateurs avec recommandations réelles	39 873
Produits indexés	13 454
Lignes de recommandations en base	398 730 (toutes prédictions ALS réelles)
Lignes de démarrage à froid en base	0 (servies dynamiquement depuis le fichier)
Durée d'ingestion	≈30 minutes
Durée d'entraînement	≈20 minutes (grille 6 param., rank=100)
Durée totale du pipeline	≈33 minutes
Latence de l'API (recommandations)	<5 ms (recherche par index sur user_id)

TABLE 7. Métriques système issues de l'exécution finale.

6 Déploiement

6.1 Prérequis

- Docker Engine et Docker Compose v2
- Au moins 8 Go de RAM alloués à Docker (l'entraînement utilise `-driver-memory 4g`)
- `dataset/Reviews.csv` (Amazon Fine Food Reviews, disponible sur Kaggle)

6.2 Démarrage en une commande

```
1 docker compose down -v && docker compose up --build
```

Le pipeline démarre automatiquement. Le DAG Airflow se déclenche au démarrage de la stack sans aucune intervention manuelle. Lorsque l'entraînement est terminé, les recommandations sont disponibles dans le tableau de bord.

Service	URL	Identifiants
Tableau de bord	<code>http://localhost:8501</code>	—
Airflow	<code>http://localhost:8080</code>	airflow / airflow
FastAPI docs	<code>http://localhost:8000/docs</code>	—
Spark UI	<code>http://localhost:8090</code>	—
PostgreSQL	<code>localhost:5433</code>	bigdata / bigdata

TABLE 8. URLs des services après démarrage de la stack.

7 Limites

Absence de noms de produits officiels. `Reviews.csv` ne contient aucune colonne de nom de produit. Les noms d'affichage dans le tableau de bord sont dérivés du résumé de l'évaluateur le plus utile (ex. : « Good Quality Dog Food »). Il s'agit de formulations rédigées par les évaluateurs, et non de titres officiels. L'ASIN reste le véritable identifiant produit.

Partition Kafka unique. Une seule partition limite la consommation parallèle. Passer à 3 partitions ou plus permettrait à Spark de paralléliser la lecture.

Absence de mise à jour en ligne du modèle. L'ALS se réentraîne intégralement à chaque déclenchement du DAG. Entre deux exécutions, le modèle est obsolète pour les nouveaux utilisateurs.

Rejeu du point de reprise. L'exécution de `docker compose down -v` efface le point de reprise du Spark Structured Streaming. Le service de streaming rejoue tous les messages Kafka depuis l'offset 0 au prochain démarrage. En production, le checkpoint devrait être stocké en externe (S3, GCS).

Légère fuite de vocabulaire. Le vocabulaire du `StringIndexer` est ajusté sur l'ensemble complet des données filtrées. L'impact est mineur — l'encodeur ne porte aucune statistique de notation — mais techniquement, les identifiants de l'ensemble de test ont été vus avant le découpage.

8 Perspectives d'amélioration

Modèle. Le filtrage collaboratif neuronal (NeuMF, LightGCN) surpasse l'ALS sur des données creuses. L'ajout de caractéristiques de contenu (catégorie produit, embeddings du texte des avis) améliorerait la qualité du démarrage à froid. L'apprentissage incrémental permettrait des mises à jour sans réentraînement complet.

Pipeline. Passer Kafka à 3 partitions ou plus et déplacer le stockage du checkpoint vers S3 améliorerait la résilience. L'ajout d'assertions de qualité de données avant le démarrage de l'entraînement permettrait de détecter les problèmes en amont. Remplacer le `LocalExecutor` par le `KubernetesExecutor` fournirait une isolation correcte des tâches.

Évaluation. Le RMSE sur les résidus mesure la précision de reconstruction, non la qualité du classement. L'ajout du NDCG@K, Précision@K et Rappel@K donnerait une image plus claire de l'utilité réelle des recommandations.

9 Contributions

El Gorrim Mohamed — Ingénieur Machine Learning

Responsable du cœur machine learning du projet. A identifié le problème de biais positif dans le jeu de données via une analyse exploratoire et proposé l'approche de décomposition des biais. A implémenté `train.py` : le pipeline complet de prétraitement (filtre d'utilité, déduplication, filtre d'activité), la logique de décomposition des biais (découpage avant calcul des biais) et la boucle d'entraînement ALS via le `TrainValidationSplit` de Spark MLlib. A identifié et corrigé le problème de fuite de données. A conçu et implémenté la logique d'*upsert* pour l'écriture des recommandations de tous les utilisateurs dans PostgreSQL via le patron de table de transit. A implémenté le stockage des biais dans PostgreSQL pour permettre des décompositions de scores réelles dans le tableau de bord.

Kchibal Ismail — Ingénieur Data

Responsable des couches d'ingestion et de streaming. A configuré le broker Kafka et le cluster ZooKeeper via Docker Compose, paramétré la création du topic via `kafka-init` et implémenté

`producer.py` pour diffuser `Reviews.csv` dans Kafka à pleine vitesse. A implémenté `stream.py` : le job Spark Structured Streaming qui applique le modèle ALS figé aux événements Kafka entrants par micro-lots de 30 secondes et effectue des *upserts* dans PostgreSQL. A conçu l'architecture de démarrage à froid : les utilisateurs inconnus sont exclus de la base et servis dynamiquement par l'API depuis `popular_products.json`. A géré les vérifications de santé Docker Compose et l'ordonnancement des dépendances entre services.

Uthman Junaid — Ingénieur Full-Stack

Responsable des couches de présentation et d'orchestration. A conçu et implémenté le DAG Airflow (`recommendation_pipeline`) : chaîne séquentielle de 4 tâches avec délais d'exécution maximum, déclenchement automatique à la création et patron producteur-en-Tâche-1 garantissant que l'entraînement démarre toujours sur un instantané Kafka complet. A développé l'application FastAPI : pool de connexions, tous les endpoints REST, le flux en direct par scrutation (remplaçant les SSE pour éliminer les crashes mémoire du navigateur) et l'endpoint de surveillance de santé avec comptage réel des messages Kafka. A développé le tableau de bord : SPA à cinq sections avec flux Kafka en direct, cartes de recommandations avec décomposition explicable du score, analytique modèle avec historique d'entraînement, EDA du jeu de données et surveillance de la santé du pipeline. A implémenté le proxy inverse Nginx.

Conclusion

Ce projet démontre qu'un prétraitement soigné transforme un algorithme classique en un système de recommandation compétitif. La contribution clé est la décomposition des biais : séparer la tendance globale, les habitudes par utilisateur et la qualité par produit du signal d'entraînement avant d'alimenter les résidus à l'ALS a réduit le RMSE de 50 à 65% par rapport à l'ALS brut sur les mêmes données.

L'architecture inspirée du modèle Lambda sépare clairement les responsabilités : l'entraînement batch produit un modèle figé sur un instantané historique ; l'inférence en streaming l'applique en continu aux nouveaux événements. Les deux couches partagent le même topic Kafka et le même dépôt PostgreSQL, orchestrés par un unique DAG Airflow qui exécute l'intégralité du pipeline automatiquement depuis un démarrage à froid en 33 minutes.

Le système est entièrement reproductible : une seule commande `docker compose down -v && docker compose up --build` lance les onze services, ingère le jeu de données complet, entraîne le modèle et sert des recommandations en direct via un tableau de bord de supervision — sans aucune intervention manuelle.